

```
--- backup\parcelas\parcelasP00.py ---
```

```
from psycopg.rows import dict_row
#from myLib.connect import connect
from myLib.gestor import tablasP00
```

```
EPSG_CODE = 25830
```

```
class ParcelasP00(tablasP00):
```

```
    # INSERT
```

```
    def insert(self, d: dict):
```

```
        wkt = d['geom_wkt']
```

```
        if not self.geom_isValid(wkt):
```

```
            print("Error: La geometría no es válida (Self-intersection, etc.)")
```

```
            self.disconnect()
```

```
            return None
```

```
        if self.has_interior_intersection('parcelas', wkt):
```

```
            print("Error: La parcela interseca el interior de una parcela existente.")
```

```
            self.disconnect()
```

```
            return None
```

```
        geom_final = self.snap_to_grid(wkt)
```

```
        dueno = d['dueno']
```

```
        area_m2 = d['area_m2']
```

```
        cultivo = d['cultivo']
```

```
        fecha_siembra = d['fecha_siembra']
```

```
        cons = """
```

```
        INSERT INTO parcelas
```

```
            (dueno, area_m2, cultivo, fecha_siembra, geom)
```

```
            VALUES (%s, %s, %s, %s, st_geometryFromText(%s, %s))
```

```
            RETURNING id
```

```
        """
```

```
        try:
```

```
            self.cur.execute(cons, [dueno, area_m2, cultivo, fecha_siembra, geom_final,
self.epsg])
```

```
            self.conn.commit()
```

```
            l = self.cur.fetchall()
```

```
            print(f"Parcela de {d['dueno']} insertada con id: {l[0][0]}")
```

```
            self.disconnect()
```

```
            return l[0][0]
```

```
        except Exception as e:
```

```
            print(f"Error en inserción: {e}")
```

```
            self.conn.rollback()
```

```
        finally:
```

```
            self.disconnect()
```

```

# SELECT
def select(self, d: dict, asDict=True):
    id_min = d['id_min']

    if asDict:
        self.cur = self.conn.cursor(row_factory=dict_row)

    cons = """
SELECT id, dueno, area_m2, cultivo, fecha_siembra, st_astext(geom) AS geom_wkt
FROM parcelas WHERE id > %s
"""
    self.cur.execute(cons, [id_min])
    l = self.cur.fetchall()
    print(f"{len(l)} parcela(s) encontrada(s).")
    self.disconnect()
    return l

# UPDATE
def update(self, d: dict):
    id = d['id']
    dueno = d['dueno']
    area_m2 = d['area_m2']
    cultivo = d['cultivo']
    fecha_siembra = d['fecha_siembra']
    geom_wkt = d['geom_wkt']

    cons = """
UPDATE parcelas
SET
    dueno = %s,
    area_m2 = %s,
    cultivo = %s,
    fecha_siembra = %s,
    geom = st_geometryFromText(%s, %s)
WHERE
    id = %s
"""
    self.cur.execute(cons, [dueno, area_m2, cultivo, fecha_siembra,
                           geom_wkt, EPSG_CODE, id])
    print(f"Filas actualizadas: {self.cur.rowcount}")
    self.conn.commit()
    self.disconnect()

# DELETE
def delete(self, d: dict):
    id = d['id']

    cons = """
DELETE FROM parcelas
WHERE id = %s
"""
    self.cur.execute(cons, [id])
    print(f"Filas eliminadas: {self.cur.rowcount}")

```

```
self.conn.commit()  
self.disconnect()
```